

Delta SPMU Academy

E-learning platform for permanent-makeup (SPMU) training — blended model: online theory via a Frappe LMS backend + in-person practical training. Based in Addis Ababa, Ethiopia.

Generated 2026-07-15 · Reflects `main` @ current working tree · Status: staging live, prod live

Contents

1. System context

2. Technology stack

3. Frontend architecture

4. Backend architecture

5. Data & domain model

6. Payments architecture
7. Video architecture

8. Auth, sessions & security

9. Environments

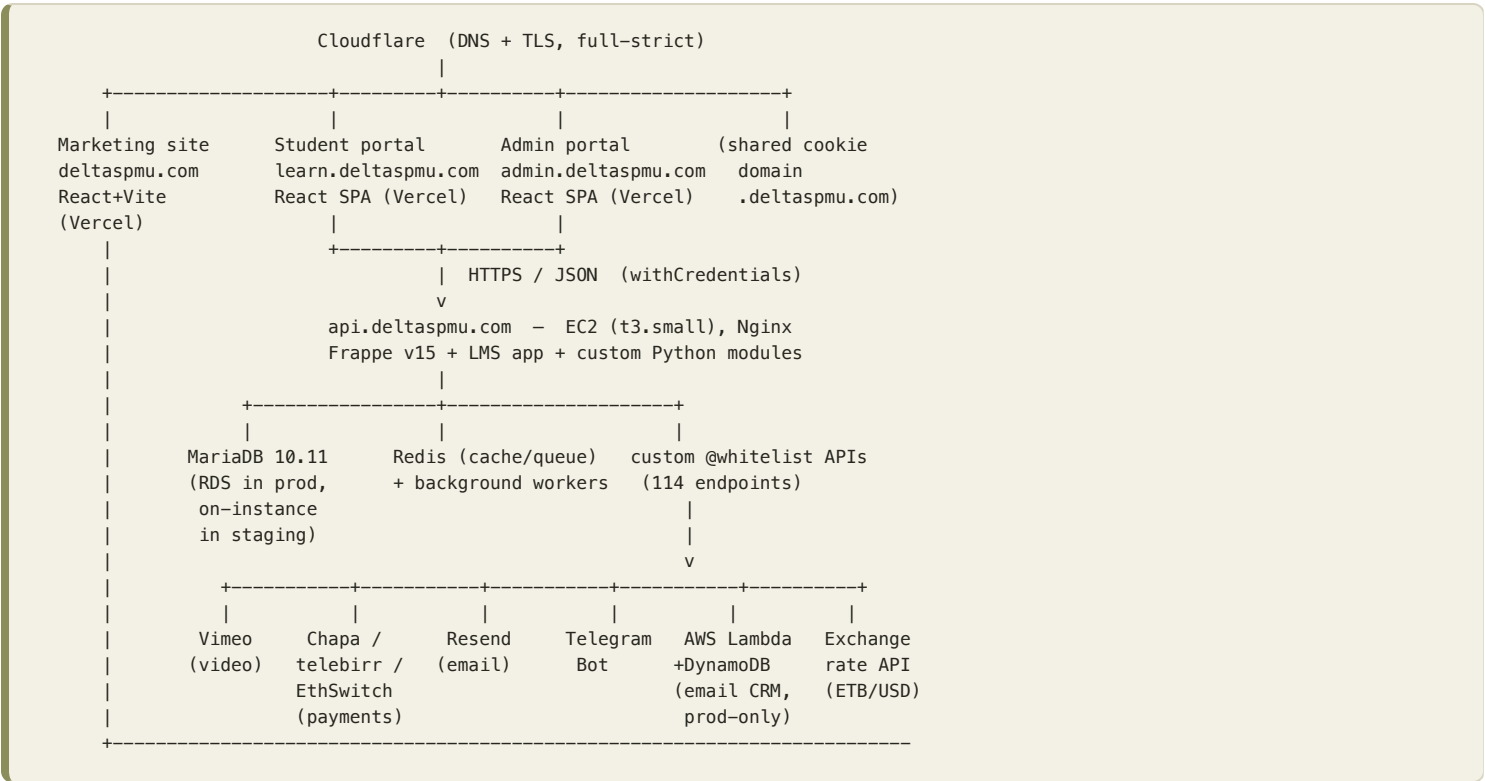
10. Infrastructure (Terraform)

11. Key data flows

12. Branching & delivery

1 • System context

Three separately-hosted frontends talk to one Frappe backend over HTTPS. DNS/TLS is fronted by Cloudflare. The architecture mirrors the production Afritutors platform.



2 • Technology stack

Layer	Technology	Notes
Frontend framework	React 19 + TypeScript + Vite	All three apps
Styling	Tailwind CSS	Marketing: v3; portals: v4
Server state	TanStack React Query 5	Portals
Auth state	React Context	Portals
Forms / validation	react-hook-form + zod	Student portal
Rich text	TipTap 3	Admin course editor
i18n	i18next (EN + AM)	Locale files complete; AM not yet rendered
Backend	Frappe v15 (Python) + LMS app fork	Custom overlay modules
Database	MariaDB 10.11	RDS (prod) / on-instance (staging) / Docker (dev)
Cache / queue	Redis	Sessions + background jobs
Compute	AWS EC2 t3.small (eu-central-1)	Nginx reverse proxy
Video	Vimeo (per-env tag)	Resumable upload via tus
Frontend hosting	Vercel	Auto-deploy from GitHub; all 3 apps
DNS + TLS	Cloudflare	Full-strict
Email (transactional)	Resend	Mailpit locally in dev
Email (CRM)	AWS Lambda + DynamoDB + API Gateway	Prod-only; inert on staging
Payments	Chapa, telebirr, EthSwitch, CBE	Chapa live; others staged
Messaging	Telegram Bot	Admin broadcast
IaC	Terraform (S3 remote state)	envs/{staging,prod} + shared modules

3 · Frontend architecture

Three independent Vite apps, deployed to Vercel. All portal requests use `axios` with `withCredentials:true` so Frappe's session cookie rides along. `localStorage` keys use the `deltaspmu_` prefix.

3.1 Marketing site — `deltaspmu.com`

Root of the repo (`src/` , `index.html`). React 19 + Vite + **Tailwind 3**, `framer-motion` for animation and `lenis` for smooth scrolling. Pure marketing: course overview, pricing/bundle, FAQ, contact — deep-links into the student portal to enrol. Served by Vercel (a legacy S3+CloudFront pipeline exists but is unused).

Branding Olive + terracotta on warm cream. Tokens in `tailwind.config.js` ; the legacy `nude` token resolves to terracotta for backward compatibility.

3.2 Student portal — `learn.deltaspmu.com`

`frontend/student-portal/` — 27 route pages. Single API surface in `src/api/client.ts` . Key libraries: TanStack Query, react-hook-form + zod, `@vimeo/player` , `dompurify` (sanitises lesson HTML), react-helmet-async, react-hot-toast, react-i18next.

Area	Pages
Catalog	Courses, CourseDetail, Wishlist
Auth	Register, Login, VerifyEmail, ForgotPassword, ResetPassword
Commerce	Payment, PaymentSuccess, Transactions
Learning	Learn / LearnCourse / LearnLesson, Quiz / QuizPage
Account	Dashboard, MyCourses, Profile, Certificates
Static / legal	About, Help, Contact, Terms, Privacy, Refund, Cookies

3.3 Admin portal — `admin.deltaspmu.com`

`frontend/admin-portal/` — 24 pages, System-Manager-gated. Three API clients: `client.ts` (Frappe), `emailClient.ts` (Lambda CRM), `vimeo.ts` (resumable upload via `tus-js-client`). Course body editing uses TipTap.

Cross-origin CSRF: the admin portal cannot read the `csrf_token` cookie cross-origin, so it fetches a token from the `get_csrf_token` endpoint and caches it for POST/PUT/DELETE.

Area	Pages
Overview	Dashboard, Analytics
Catalog mgmt	CourseList, CourseForm, CourseEdit, CourseEditor, Categories
Assessment	Quizzes, Certificates, Reviews
People	Users / UserList, Enrollments
Commerce	Payments, Settings
Media & comms	Videos, TelegramBroadcast, Email* (Inbox, Compose, List, Detail, Addresses)

4 · Backend architecture

A forked Frappe LMS app with a custom Python overlay. Files deploy to `/home/frappe/deltaspmu/apps/lms/lms/lms/`. **114** whitelisted endpoints across 16 modules.

Module	Responsibility
<code>api.py</code>	Core: auth, courses, enrolment, profiles, branding, catalog
<code>custom_api.py</code>	Learning progress, quiz grading, certificates, progress reports
<code>payments_api.py</code>	Payment orchestration: initiation, verification, webhooks, bundle pricing
<code>chapa.py</code>	Chapa gateway integration (live)
<code>telebirr.py</code> , <code>telebirr_c2b.py</code> , <code>telebirr_c2b_xml.py</code>	telebirr (RSA-PSS, C2B flows)
<code>ethswitch.py</code>	EthSwitch gateway
<code>vimeo_api.py</code>	Server-side Vimeo proxy; per-env <code>vimeo_tag</code> resolution
<code>exchange_rate.py</code>	ETB/USD conversion with live rate + fallback chain
<code>security.py</code>	Rate limiting, user ban/unban
<code>user.py</code>	User lifecycle hooks (verification enable/disable)
<code>telegram_bot.py</code>	Telegram broadcast + link status
<code>email_templates.py</code>	Transactional email templates (Resend)
<code>course_import_export.py</code>	prod→staging/dev course-content sync (carries custom fields/doctypes)
<code>_cert_backfill.py</code> , <code>_migrate_doctypes.py</code>	One-off migration/backfill utilities

4.1 Backend conventions (load-bearing)

- **Never** `frappe.get_doc()` in guest endpoints — it raises `PermissionError` for anonymous callers. Use `frappe.db.get_value()` / `get_list()` instead.
- Course price field is `course_price` (not `price`); default fallback applies.
- **CSRF** is required on POST/PUT/DELETE; GET bypasses it — payment callbacks use GET intentionally. (Note: `ignore_csrf` is currently enabled across envs — see §13.)
- Sessions are cookie-based → the frontend must send credentials on every request.
- After adding `@frappe.whitelist()` methods, clear `.pyc` cache and `bench restart`.
- The **"all 5 courses" bundle** is a *virtual* product (no LMS Course row): `initiate_payment` and `get_course_price` special-case `all-courses-bundle` at 20,000 ETB (per-course 12,500 ETB).
- Transaction IDs are prefixed `DS-`.

5 · Data & domain model

Persistence is Frappe doctypes on MariaDB. The domain centres on the course→chapter→lesson tree, enrolment/access, assessment, and commerce.

Concept	Backing / notes
Course	LMS Course (custom field <code>course_price</code> , learning outcomes); published flag
Chapter → Lesson	Curriculum tree; ~10 chapters / 28 lessons per flagship course; Vimeo id per lesson
Category	Course categorisation (catalog filter)
Enrolment / Access	Manual + paid; 365-day access window; progress %
Quiz / Question / Attempt	Per-lesson gating quizzes + final quiz; server-side grading; attempt limits
Certificate	Auto-issued on 100% completion; id format <code>DELTA-SPMU-xxxx</code> ; PDF via <code>get_certificate_pdf</code>
Payment / Transaction	Gateway records, status, method; <code>DS-</code> prefixed ids
User	Frappe User; disabled until email verification; roles gate portals

Content model note: Course catalog and pricing are seeded/synced from prod and are not uniform (flagship 17,000 ETB; others 12,500 ETB), which differs from the "each course 12,500 ETB" baseline in project docs.

6 • Payments architecture

`payments_api.py` is the orchestrator; each gateway has its own module. Prices are computed server-side (`get_course_price`) with live ETB/USD conversion and bundle special-casing.

```
Student → Payment page → initiate_payment (GET) ┌
                                                    ├── Chapa (live, test keys on staging)
                                                    ├── telebirr (RSA-PSS; staged)
                                                    ├── EthSwitch (staged)
                                                    └── CBE (manual verification)
gateway hosted checkout → webhook/callback (GET, CSRF-exempt)
→ verify → create Course Access → enrol → grant learning access
```

- Only **Chapa** is fully wired (test keys live on staging); telebirr / EthSwitch are deferred; CBE is manual transfer verification.
- Gateway rejects a request (e.g. reserved-domain email) → surfaces as a payment error to the UI.
- The bundle is virtual — no LMS Course row; priced in code.

7 • Video architecture

Videos live on Vimeo and are proxied server-side so the API token never reaches the browser. Each lesson stores `"id/hash"` (split on `/` for id + privacy hash).

Per-environment isolation: `vimeo_api._get_tag()` reads `vimeo_tag` from `site_config`, so uploads never cross environments — prod `deltaspmu-lms`, staging `deltaspmu-lms-staging`, dev `deltaspmu-lms-dev`. Admin upload is resumable via `tus-js-client`.

8 • Auth, sessions & security

- **Session model:** Frappe cookie sessions in Redis; portals send `withCredentials:true`. The cookie is scoped to `.deltaspmu.com`.
- **CSRF:** required on mutating requests; admin fetches a token cross-origin via `get_csrf_token`.
- **Guest safety:** guest endpoints avoid `get_doc()` to prevent permission errors.
- **Abuse controls:** `security.py` handles rate limiting + user banning.
- **Email verification:** new users are created disabled and enabled only after verifying via the `/verify` flow.

9 • Environments

	dev	staging	prod
Compute	Local Docker Compose	Single EC2 + on-instance MariaDB	EC2 + RDS MariaDB
API host	<code>lms.localhost</code>	<code>staging-api.deltaspmu.com</code>	<code>api.deltaspmu.com</code>
Portals	local Vite	<code>staging-learn / staging-admin</code> (branch <code>staging</code>)	<code>learn / admin</code>
Email	Mailpit (<code>:8025</code>)	Resend	Resend + Lambda CRM
Video tag	<code>deltaspmu-lms-dev</code>	<code>deltaspmu-lms-staging</code>	<code>deltaspmu-lms</code>
Payments	—	Chapa (test)	Chapa (live) + others
Marketing/CDN	—	not deployed	Vercel (S3/CDN legacy, unused)

Dev backend edits sync via `./scripts/dev-sync-backend.sh` ; the Docker stack is `mariadb 10.11` , `redis` (cache + queue), `frappe/bench` , and `mailpit` . Secrets live in `~/.deltaspmu/staging-keys.env` and are never committed.

10 • Infrastructure (Terraform)

Region `eu-central-1` . Roots at `infrastructure/envs/{staging,prod}` compose shared `infrastructure/modules/` . Remote state in S3 `deltaspmu-tfstate-*` . AWS resources are named `deltaspmu-dev-*` (legacy naming, accepted debt).

Module	Purpose	staging	prod
<code>network</code>	VPC, subnets, security groups	✓ (no DB SG)	✓
<code>backend-server</code>	EC2, EIP, optional on-instance MariaDB	✓ (MariaDB on-box)	✓
<code>rds</code>	Managed MariaDB 10.11	—	✓
<code>marketing</code>	S3 + CloudFront (marketing)	—	✓ (unused)
<code>email</code>	Lambda + DynamoDB + API Gateway (CRM)	—	✓ (placeholder)

Guardrail: never `terraform apply` in `envs/prod` without a clean plan and explicit approval. Staging is cost-optimised: no RDS, no email stack, no CDN.

11 · Key data flows

11.1 Learn-and-certify (verified end-to-end)

```
register → verify email (enable user) → login → browse catalog
  → buy (gateway) OR admin manual enrol → Course Access (365 days)
  → learn: lessons render, Vimeo embed, mark_lesson_complete → progress %
  → per-lesson gating quizzes + final quiz (server-graded, attempt-limited)
  → 100% lessons + all quizzes passed → certificate auto-issued → PDF
```

11.2 Admin content lifecycle

```
create/clone course → TipTap body + curriculum builder (chapters/lessons)
  → resumable Vimeo upload (tus) → attach video ids → Save Draft ≠ Publish
  → manage quizzes, categories, enrolments, reviews, payments, Telegram broadcast
```

12 · Branching & delivery

Changes are promoted through environments — never pushed straight to prod.

```
feature/bugfix/hotfix branch → staging → main (prod)
  (local dev)                (verify)   (release)
```

- Never commit directly to `main` or `staging` ; both advance only via merged PRs.
- Work on `feature/*` , `bugfix/*` , or `hotfix/*` off `staging` .
- Open a PR against `staging` ; tests must pass and the change is verified on staging before merge.
- Promote by merging `staging → main` ; deploying `main` is the prod release.
- Frontends auto-deploy from GitHub via Vercel; backend files deploy with `./scripts/deploy-backend.sh {staging|prod}` .